

GMS · GYM MANAGEMENT SYSTEM

Estrategia de Testing & Integración Continua

Documentación técnica de pruebas unitarias, de integración,
E2E y configuración de CI con GitHub Actions

FECHA

Mayo 2026

RAMA

dev → main

TEST RUNNER

Vitest 4 + Playwright

TOTAL TESTS

65 unit + E2E

1. Resumen Ejecutivo

65

Tests unitarios & de integración

7

Archivos de test (unit + integration)

~1.4s

Tiempo de ejecución tests unitarios

2

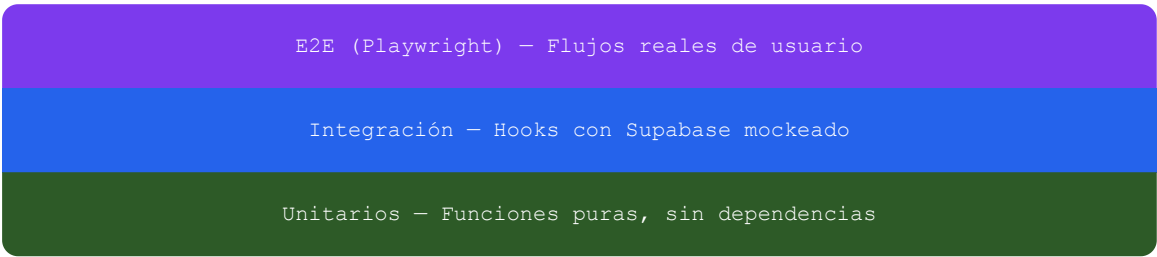
Workflows de CI GitHub Actions

La estrategia de testing del proyecto GMS está organizada en tres capas independientes que cubren distintos niveles de la aplicación. Cada capa tiene su propia herramienta, su propio entorno de ejecución y su propio rol dentro del pipeline de CI/CD.

CAPA	TIPO	HERRAMIENTA	ENTORNO	TESTS	TIEMPO APROX.
Funciones puras	Unitario	Vitest 4	Sin dependencias externas	48	~0.5s
Hooks React	Integración	Vitest + Testing Library	Supabase mockeado	17	~1s
Flujos de usuario	E2E	Playwright	App + Supabase real	En expansión	~5-10min

2. Pirámide de Testing

La distribución de tests sigue la pirámide clásica: muchos tests rápidos y baratos en la base, pocos tests lentos y costosos en la cima. Esto garantiza feedback rápido en desarrollo sin sacrificar cobertura real de los flujos críticos.



Principio central: los tests de integración y unitarios no requieren ninguna credencial, servidor ni variable de entorno. Todo lo que depende de Supabase está reemplazado por mocks, lo que permite ejecutarlos en cualquier entorno — incluyendo CI— de forma completamente autónoma.

3. Estructura de Archivos

Los tests unitarios y de integración están colocados junto a los archivos fuente que prueban (*colocated tests*). Los tests E2E viven en su propio directorio separado.

```
gms-app/
├── .github/workflows/
│   ├── unit-tests.yml ← CI: tests unitarios (nuevo)
│   └── e2e.yml ← CI: tests E2E (existente)
│
├── vitest.config.ts ← Configuración del runner (nuevo)
│
├── src/
│   ├── tests/setup.ts ← Setup global: jest-dom (nuevo)
│   │
│   ├── lib/
│   │   ├── authUtils.ts ← Funciones puras extraídas de useAuth (nuevo)
│   │   ├── authUtils.test.ts ← 15 tests unitarios (nuevo)
│   │   ├── sociosUtils.ts ← Funciones puras extraídas de useSocios (nuevo)
│   │   ├── sociosUtils.test.ts ← 19 tests unitarios (nuevo)
│   │   ├── discountCalc.ts ← existente
│   │   ├── discountCalc.test.ts ← 11 tests unitarios (nuevo)
│   │   ├── freezeCalc.ts ← existente
│   │   └── freezeCalc.test.ts ← 3 tests unitarios (existente)
│   │
│   ├── hooks/
│   │   ├── useAuth.ts ← refactorizado: importa de authUtils
│   │   ├── useAuth.test.ts ← 6 tests de integración (nuevo)
│   │   ├── useSocios.ts ← refactorizado: importa de sociosUtils
│   │   ├── useSocios.test.ts ← 6 tests de integración (nuevo)
│   │   ├── useAltaSocio.ts ← existente
│   │   └── useAltaSocio.test.ts ← 5 tests de integración (nuevo)
│   │
│   └── e2e/ ← Playwright (separado, existente)
│       ├── tests/auth/login.spec.ts
│       └── tests/socios/nuevo-socio.spec.ts
```

Refactoring realizado para habilitar tests

Las funciones puras que existían como closures dentro de los hooks fueron extraídas a archivos de librería independientes. Esto no cambia el comportamiento de la aplicación, pero permite probarlas de forma aislada sin necesidad de montar el hook completo.

FUNCIÓN	ORIGEN (ANTES)	DESTINO (AHORA)	EXPORTADA
readCache()	useAuth.ts (privada)	authUtils.ts	✓

FUNCIÓN	ORIGEN (ANTES)	DESTINO (AHORA)	EXPORTADA
writeCache()	useAuth.ts (privada)	authUtils.ts	✓
clearCache()	useAuth.ts (privada)	authUtils.ts	✓
isSoftExpired()	useAuth.ts (privada)	authUtils.ts	✓
isInactivityExpired()	useAuth.ts (privada)	authUtils.ts	✓
resolveStatus()	useSocios.ts (closure)	sociosUtils.ts	✓
calcDiasRestantes()	useSocios.ts (closure)	sociosUtils.ts	✓
calcHasDeuda()	useSocios.ts (closure)	sociosUtils.ts	✓
buildInitials()	useSocios.ts (closure)	sociosUtils.ts	✓
formatDateAR()	useSocios.ts (closure)	sociosUtils.ts	✓

4. Tests Unitarios

Los tests unitarios prueban funciones puras de forma completamente aislada. No requieren mocks, no montan componentes React, no conectan a ningún servicio externo. Dado un input, validan el output esperado.

4.1 authUtils.test.ts — 15 tests

Cubre todas las funciones de caché de sesión y verificación de expiración.

SUITE	TEST	QUÉ VERIFICA
readCache	retorna null sin caché	localStorage vacío → null
	retorna usuario con JSON válido	deserialización correcta
	retorna null con JSON corrupto	manejo silencioso de errores
writeCache / clearCache	writeCache persiste y readCache recupera	ciclo completo de caché
	clearCache elimina usuario y login_ts	limpieza total de localStorage
isSoftExpired	false sin límite configurado	caso base: sin configuración
	false dentro del límite de horas	sesión vigente
	true cuando superó el límite	sesión expirada
	false sin login_ts guardado	caso de primer login
isInactivityExpired	false sin límite configurado	caso base: sin configuración
	false dentro del límite	usuario activo recientemente
	true cuando superó el límite	inactividad prolongada

4.2 sociosUtils.test.ts — 19 tests

SUITE	TEST	QUÉ VERIFICA
resolveStatus	CANCELADA sin membresía	null → CANCELADA
	ACTIVA con vencimiento futuro	status se respeta
	EN_GRACIA cuando end_date ya pasó	fallback hasta que el proceso BD actualice
	CONGELADA sin importar fecha	status explícito prevalece
	IMPAGO directo desde status	pass-through de status almacenado
calcDiasRestantes	-1 sin membresía	caso nulo
	-1 sin end_date	membresía sin fecha de vencimiento
	0 cuando ya venció	no retorna negativos

SUITE	TEST	QUÉ VERIFICA
	días aproximados con vencimiento futuro	cálculo matemático correcto
calcHasDeuda	false sin membresía	caso nulo
	false sin pagos	membresía nueva sin cobrar
	false con PAGO_TOTAL	pago completo no genera deuda
	true con CUOTA_1 sin CUOTA_2	deuda real detectada
	false con CUOTA_1 y CUOTA_2	deuda saldada
buildInitials	JP de "Juan Pérez"	dos palabras
	A de "Admin"	una sola palabra
	ME de "María Elena Torres"	ignora tercera palabra
formatDateAR	"—" cuando fecha es null	valor centinela
	DD/MM/YYYY en locale es-AR	formato regional correcto

4.3 discountCalc.test.ts — 11 tests

SUITE	TEST	QUÉ VERIFICA
Factor continuidad	0% con menos de 91 días	socio nuevo sin descuento
	40% del alloc entre 91 y 364 días	descuento parcial
	100% del alloc con 365+ días	descuento máximo por fidelidad
Factor volumen	0% con plan menor a 90 días	sin beneficio por plan corto
	40% del alloc entre 90 y 179 días	descuento parcial
	100% del alloc con 180+ días	descuento máximo por plan largo
Factor nivel	0% para BASICO	plan básico sin bonus
	40% para SILVER y GOLD	descuento intermedio
	100% para VIP y PREMIUM	descuento máximo
Cap máximo	final nunca supera discount_cap	techo de descuento respetado
	capped=false cuando no supera	flag correcto

4.4 freezeCalc.test.ts — 3 tests (preexistente)

- ✓ Descongelamiento el mismo día → 0 días usados, end_date vuelve al original
- ✓ Descongelamiento anticipado (7 de 14 días) → end_date = original + 7
- ✓ Descongelamiento tardío → cap en freeze_days, end_date sin cambio

5. Tests de Integración

Los tests de integración prueban los hooks de React junto con su lógica interna, pero reemplazan las llamadas reales a Supabase con funciones simuladas (*mocks*). Esto permite verificar que los hooks responden correctamente a cada escenario sin necesidad de una base de datos real.

Estrategia de mock: se utiliza `vi.mock('../lib/supabase', ...)` para reemplazar el cliente de Supabase por completo. Las funciones del mock se declaran con `vi.hoisted()` para garantizar que estén disponibles en el momento en que Vitest hace el hoisting de `vi.mock` al inicio del archivo.

5.1 useAuth.test.ts — 6 tests

Módulos mockeados: `supabase.auth.onAuthStateChange`, `supabase.auth.signOut`, `supabase.auth.getSession`, `supabase.from`

TEST	ESCENARIO	VERIFICA
Estado inicial sin caché	localStorage vacío al montar	loading=true, user=null
Estado inicial con caché	usuario serializado en localStorage	loading=false, user hidratado
Evento SIGNED_IN exitoso	Supabase retorna datos de usuario	user seteado, loading=false
Evento SIGNED_OUT	sesión cerrada	user=null, caché limpiado
Sesión vacía en INITIAL_SESSION	sin usuario en la sesión	user=null, loading=false
Soft-timeout en INITIAL_SESSION	login_ts indica sesión expirada	signOut() llamado 1 vez

5.2 useSocios.test.ts — 6 tests

Módulo mockeado: `supabase.from('users').select().eq().order()`

TEST	ESCENARIO	VERIFICA
Estado inicial	al montar antes de recibir datos	loading=true, socios=[]
Transformación completa	socio con membresía activa y PAGO_TOTAL	nombre, iniciales, status, plan, sede, diasRestantes, hasDeuda=false
Deuda detectada	solo CUOTA_1 sin CUOTA_2	hasDeuda=true
Sin perfil (solo email)	socio_profiles es null	nombre = email
Sin membresías	memberships=[]	status=CANCELADA, membershipId=null
Error de Supabase	query retorna error	error seteado, socios=[]
Membresía más reciente	múltiples membresías con fechas distintas	se selecciona la de start_date más reciente

5.3 useAltaSocio.test.ts — 5 tests

Módulo mockeado: `supabase.functions.invoke('crear-socio', ...)`

TEST	ESCENARIO	VERIFICA
Estado inicial	hook recién montado	loading=false, error=null
Alta exitosa	edge function retorna ok:true + user_id	retorna userId, loading=false, error=null
Error de negocio	edge function retorna ok:false	retorna null, error seteado con mensaje
Error de red	invoke retorna error de transporte	retorna null, error seteado
resetError	luego de un error previo	error=null

6. Tests E2E con Playwright

Los tests End-to-End simulan el comportamiento real de un usuario en el navegador, interactuando con la interfaz gráfica y la base de datos de Supabase en un ambiente real. Son los más lentos pero los que mayor confianza dan sobre el funcionamiento completo del sistema.

6.1 Estructura del proyecto E2E

```
e2e/
├─ playwright.config.ts ← configuración base
├─ .env.e2e ← variables de entorno (gitignored)
├─ .auth/ ← estados de sesión por rol
│ └─ dueno.json
│ └─ encargado.json
│ └─ staff.json
│ └─ entrenador.json
│ └─ socio.json
├─ fixtures/index.ts ← fixtures globales
├─ helpers/
│ └─ auth.ts ← helper de autenticación
│ └─ supabase.ts ← helper de base de datos
├─ pages/ ← Page Object Model
│ └─ LoginPage.ts
│ └─ SociosPage.ts
│ └─ NuevoSocioWizardPage.ts
└─ tests/
    └─ auth/
        └─ login.spec.ts ← flujos de autenticación
    └─ socios/
        └─ nuevo-socio.spec.ts ← alta de socios
```

6.2 Patrón de autenticación

Los tests E2E utilizan **storage state** de Playwright para reutilizar sesiones autenticadas entre tests. Cada rol del sistema tiene su propio archivo de estado guardado, evitando el login en cada test individual.

ARCHIVO	ROL	PERMISOS RELEVANTES
dueno.json	R1_DUENO	Acceso total al sistema
encargado.json	R2_ENCARGADO	Operaciones + finanzas
staff.json	R3_STAFF	Cobros y alta de socios
entrenador.json	R4_ENTRENADOR	Solo lectura de socios asignados
socio.json	R5_SOCIO	Solo sus propios datos

6.3 Convención data-testid

Todos los elementos interactivos del UI tienen atributos `data-testid` que los tests E2E usan para seleccionar elementos de forma estable y desacoplada del CSS.

MÓDULO	PREFIJO	EJEMPLO
Autenticación	<code>login-</code>	login-input-email, login-btn-submit
Socios	<code>members-</code>	members-list, members-btn-new
Membresías	<code>memberships-</code>	memberships-page, memberships-table
Cobros	<code>payments-</code>	payments-table, payments-table-row
Wizard nuevo socio	<code>wizard-</code>	wizard-step-profile, wizard-input-firstname
Alertas	<code>alerts-</code>	alerts-list, alerts-list-item

7. Integración Continua — GitHub Actions

El repositorio tiene dos workflows independientes, cada uno optimizado para su tipo de test.

7.1 unit-tests.yml — Tests unitarios e integración

Característica clave: este workflow no requiere ningún secret, ninguna variable de entorno ni conexión a servicios externos. Es completamente autónomo.

PROPIEDAD	VALOR
Archivo	<code>.github/workflows/unit-tests.yml</code>
Runner	ubuntu-latest
Timeout	10 minutos
Secrets requeridos	Ninguno
Node version	20
Tiempo estimado	~30 segundos

Disparadores

EVENTO	RAMAS	RESULTADO
push	main, qa, dev	Corre siempre
pull_request	main, qa	Bloquea merge si falla

Pasos del workflow

1. <code>actions/checkout@v4</code>	→ clonar el repositorio
2. <code>actions/setup-node@v4</code>	→ instalar Node 20 con caché de npm
3. <code>npm ci</code>	→ instalar dependencias del <code>package-lock.json</code>
4. <code>npm run test:unit</code>	→ vitest run (65 tests, ~1.4s)
5. <code>upload-artifact@v4</code>	→ subir coverage/ si el job falla

7.2 e2e.yml — Tests E2E

PROPIEDAD	VALOR
Archivo	<code>.github/workflows/e2e.yml</code>
Runner	ubuntu-latest
Timeout	30 minutos

8. Comandos de Ejecución

COMANDO	DESCRIPCIÓN	CUÁNDO USARLO
<code>npm run test:unit</code>	Ejecuta todos los tests unitarios e integración una sola vez	Antes de hacer push, en CI
<code>npm run test:unit:watch</code>	Modo watch: re-ejecuta al guardar archivos	Durante desarrollo activo
<code>npm run test:unit:ui</code>	Abre la interfaz gráfica de Vitest en el browser	Debugging, exploración
<code>npm run test:e2e</code>	Ejecuta todos los tests E2E (requiere app corriendo)	Validación pre-release
<code>npm run test:e2e:ui</code>	Abre Playwright UI para ver tests en tiempo real	Debugging de tests E2E
<code>npm run test:e2e:headed</code>	Corre E2E con browser visible	Debugging visual
<code>npm run test:e2e:report</code>	Abre el último reporte HTML de Playwright	Post-ejecución

9. Configuración de Vitest

```
// vitest.config.ts
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom', // simula el DOM del browser
    setupFiles: ['./src/tests/setup.ts'],
    globals: true, // describe/it/expect sin imports
    include: ['src/**/*.test.ts', 'src/**/*.test.tsx'],
    exclude: ['e2e/**', 'node_modules/**'],
    coverage: {
      provider: 'v8',
      reporter: ['text', 'html'],
      include: ['src/lib/**', 'src/hooks/**'],
    },
  },
})
```

10. Dependencias de Testing Instaladas

PAQUETE	VERSIÓN	PROPÓSITO
vitest	^4.1.7	Test runner principal
@vitest/ui	^4.1.7	Interfaz gráfica de Vitest
jsdom	^29.1.1	Simulación del DOM del browser
@testing-library/react	^16.3.2	renderHook, waitFor, act
@testing-library/jest-dom	^6.9.1	Matchers de DOM (toBeInDocument, etc.)
@playwright/test	^1.60.0	Framework E2E

11. Tabla Resumen de Cobertura

MÓDULO	ARCHIVO FUENTE	TIPO DE TEST	TESTS	ESTADO
Auth — Cache & sesión	lib/authUtils.ts	Unitario	15	✓ Passing
Socios — lógica de datos	lib/sociosUtils.ts	Unitario	19	✓ Passing
Descuentos	lib/discountCalc.ts	Unitario	11	✓ Passing
Congelamiento	lib/freezeCalc.ts	Unitario	3	✓ Passing
Auth — Hook React	hooks/useAuth.ts	Integración	6	✓ Passing
Socios — Hook React	hooks/useSocios.ts	Integración	6	✓ Passing
Alta de Socio	hooks/useAltaSocio.ts	Integración	5	✓ Passing
Login UI	e2e/tests/auth/login.spec.ts	E2E	En expansión	✓ Activo
Nuevo Socio UI	e2e/tests/socios/nuevo-socio.spec.ts	E2E	En expansión	✓ Activo

65 tests  — 0 fallos

Ejecutados en 1.4 segundos · Vitest 4.1.7 · Mayo 2026